มหาวิทยาลัยมหิดล
Mahidol University

ห้ามมิให้นักศึกษาทำซ้ำ ดัดแปลง หรือใช้ประโยชน์จากงานอันมีลิขสิทธิ์ปรากฏอยู่ในระบบการศึกษาอิเล็กทรอนิกส์ (E-Learning) ของมหาวิทยาลัย ไม่ว่าจะทั้งหมดหรือบางส่วน โดยไม่ได้รับอนุญาตจากมหาวิทยาลัย นอกเหนือจากการศึกษาส่วนบุคคล ทั้งนี้ การทำซ้ำ ดัดแปลง หรือเผยแพร่ต่อสาธารณชนซึ่งงานอันมีลิขสิทธิ์ **จะมีโทษปรับตั้งแต่ 20,000 บาท ถึง 200,000 บาท และหากเป็นการกระทำเพื่อการค้า จะมีโทษจำคุกตั้งแต่ 6 เดือน ถึง 4 ปี หรือปรับตั้งแต่ 100,000 บาท ถึง 800,000 บาท หรือทั้งจำทั้งปรับ**

Unless allowed by Mahidol University, the User shall not copy, modify, or exploit in part or in whole of the copyrighted materials on the Platform, other than for your own individual study. The copy, modify or communication to public of the copyrighted materials shall be inflicted with a fine from 20,000 Baht up to 200,000 Baht. If the offence is committed with the commercial purpose, the offender shall be inflicted with imprisonment for a term from 6 months up to 4 years or a fine from 100,000 Baht up to 800,000 Baht or both imprisonment and fine.



ITDS261 Introduction to Software Engineering

Lecture 2 – Software Process Models

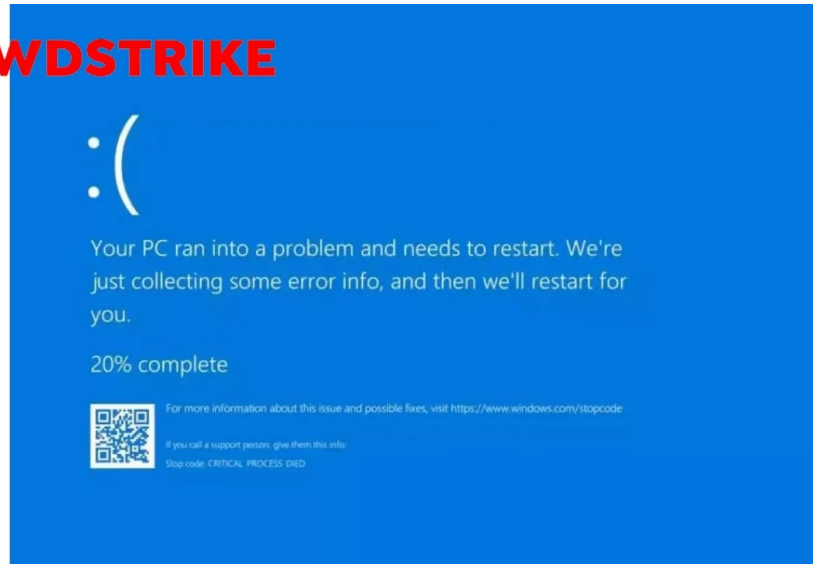
Photo: https://www.flickr.com/photos/ninara/38707088530

Topics

- Software process model definition
- Well-known software process models: Conventional vs. Modern
- Software process improvement

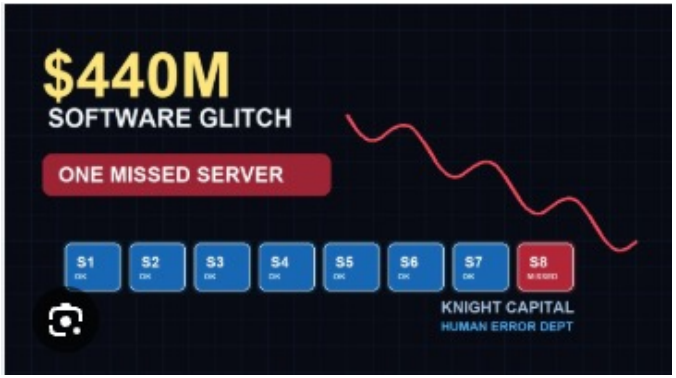
Case Study 1: CrowdStrike Outage

A bug in the Content Validator, part of CrowdStrike's deployment process, allowed an error through, the security company revealed on July 24.



References: https://www.youtube.com/watch?v=okwL2NQi_G4,
Techrepublic (2024) CrowdStrike: Buggy Validator Started Massive Outage

Case Study 2: Knight Capital



- **Knight Capital Group** caused its trading system to submit a massive number of unintended stock orders
- The faulty algorithms continued trading for approximately **45 minutes** before the system was shut down.
- The incident resulted in **more than $440 million in losses.**
- The root cause was not simply a software bug but a **failure in software deployment and operational controls**, where inadequate testing, safeguards, and risk management allowed faulty code to reach the live trading environment.

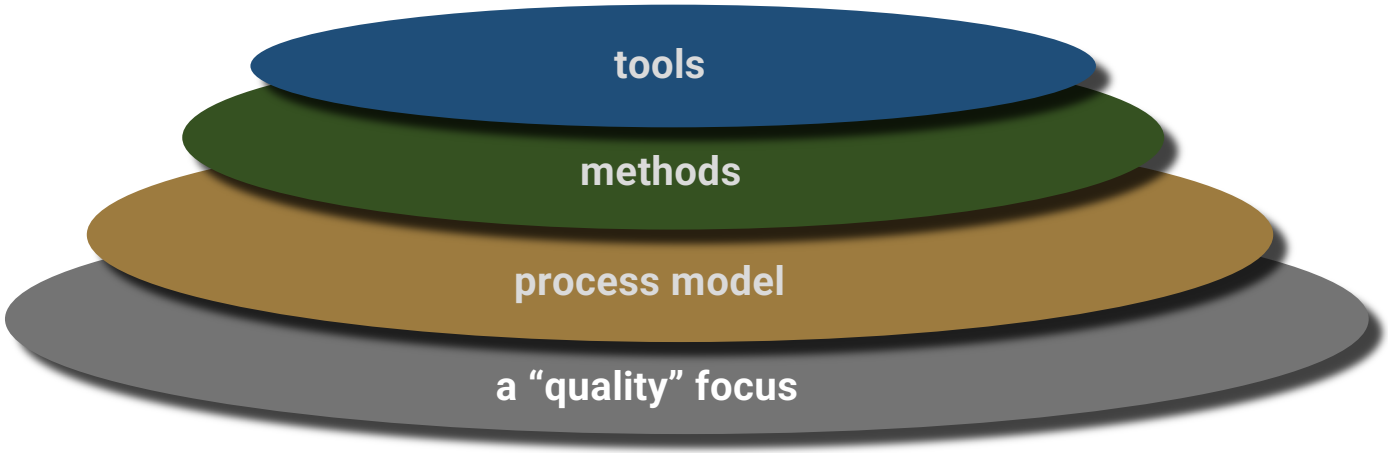
https://www.youtube.com/watch?v=PUCl_IeNNmc

https://www.henricodolfing.ch/en/case-study-4-the-440-million-software-error-at-knight-capital/

Common activities in SW development

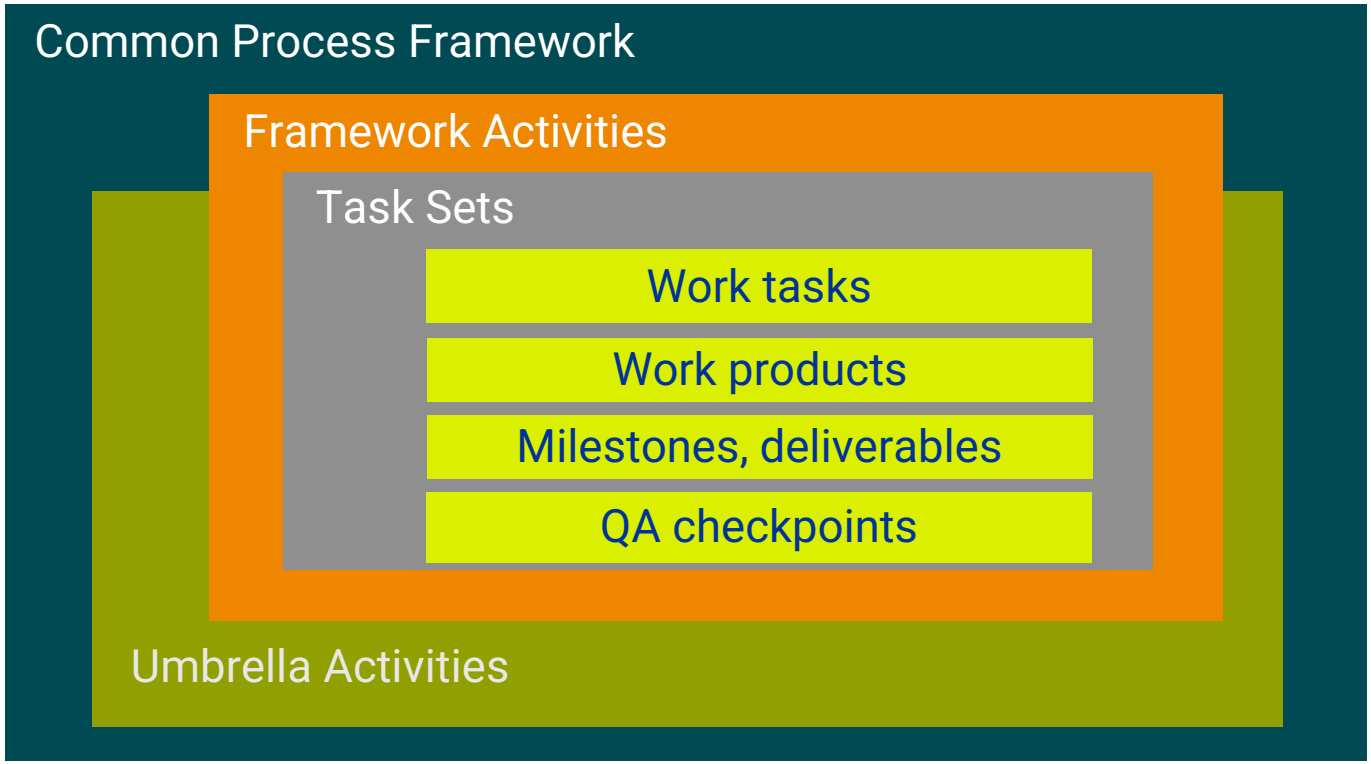
- **Requirements analysis –** Identify what the users need and define what the software should do.
- **System and software design –** Plan the structure of the system and how components will work together.
- **Implementation (coding) –** Write the program code to build the software.
- **Testing and quality assurance –** Check the software for errors and make sure it meets requirements.
- **Deployment and release –** Deliver the software for use in the real environment.
- **Maintenance and evolution –** Fix issues, update features, and adapt the software over time.

A Layered Technology



Software Engineering

A Process Framework



A Development Framework

- **Foundation:** It gives a basic structure or template for developing software, so developers don’t have to start from scratch.
- **Components and Tools:** It includes pre-built components and tools that make development faster and easier.
- **Best Practices and Guidelines:** It offers best practices and guidelines to ensure the software is built in an organized and efficient way.
- **Customization:** Developers can modify and add new functions to customize the framework to their specific needs.

Umbrella Activities

- Software project management
- Formal technical reviews
- Software quality assurance
- Software configuration management
- Work product preparation and production
- Reusability management
- Measurement
- Risk management

Process Models

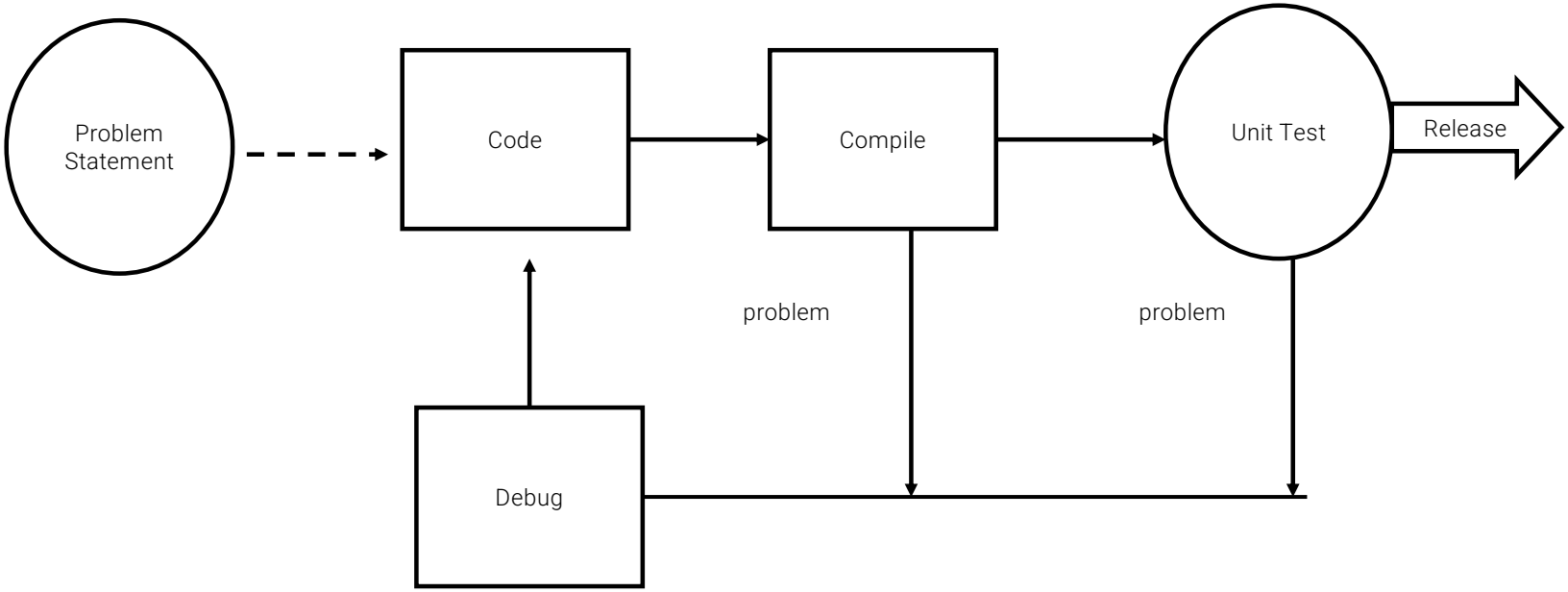
- Process models are standardised frameworks
- Provide guidance for a systematic coordination and controlling of the tasks and of the personnel who performs the tasks
- Describe the sequence of activities, roles, artifacts, and workflows involved in the development and maintenance of a software system
  - ➢ To guide software teams in planning, executing, and managing projects
  - ➢ To ensure consistency and quality of work product
  - ➢ To facilitate communication and coordination among stakeholders

*** coordination, tasks, people ***

Process Models (Cont.)

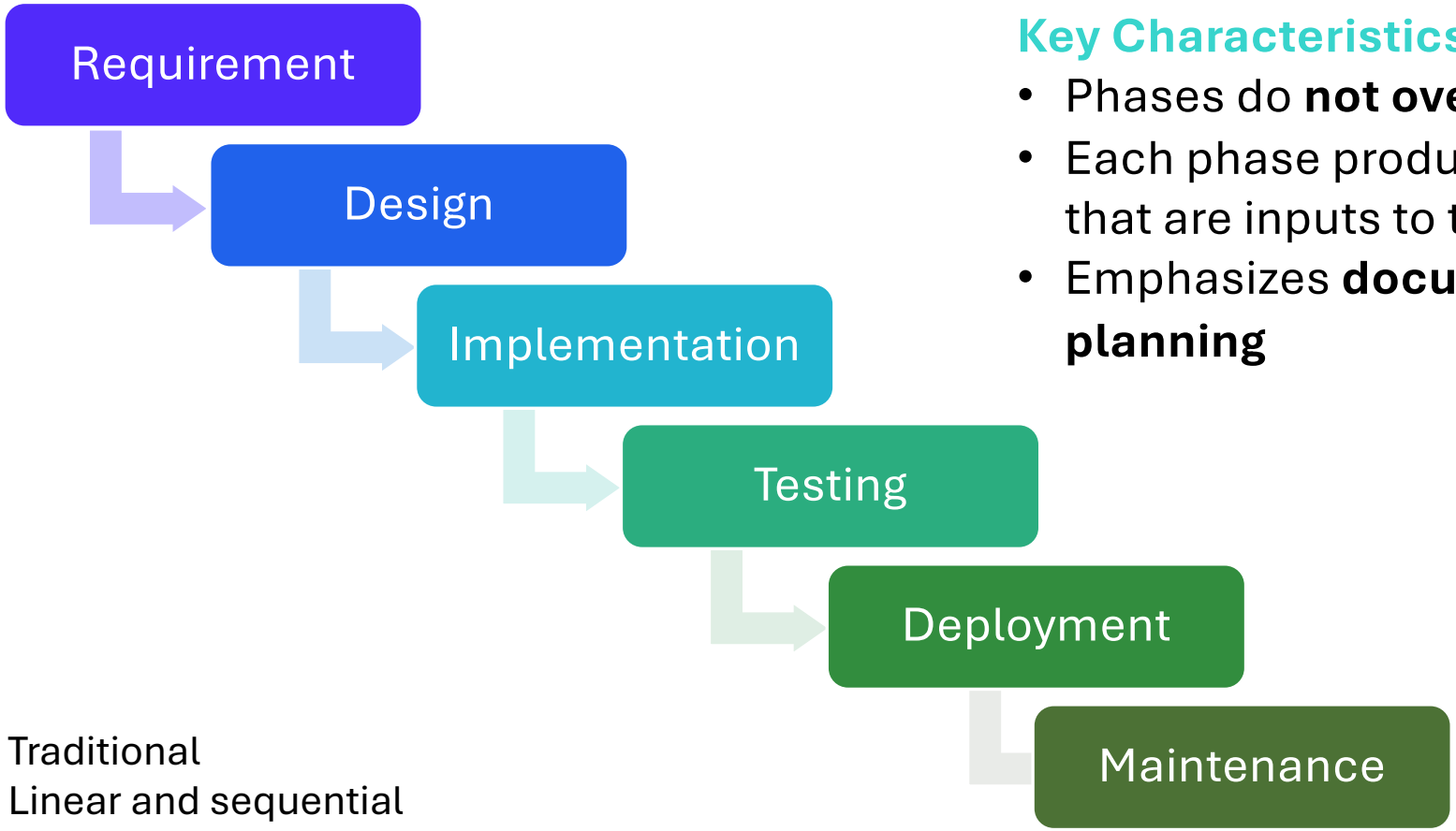
- What to define:
  - Set of tasks that need to be performed
  - Input and output of those tasks
  - Pre-conditions and post-conditions of those tasks
  - Sequence of flow of the tasks
  - Personnel who perform the tasks
- Examples:
  - Code and Fix
  - Waterfall
  - Incremental
  - Spiral
  - Agile

Code and Fix Model



1. Most people performs and follow this process, but unfortunately some *skips* unit testing or debugging
2. Also, some proceeds *without* clearly understanding the “problem statement” ---- which is requirement

Waterfall Model



Key Characteristics:

- Phases do **not overlap**.
- Each phase produces deliverables that are inputs to the next.
- Emphasizes **documentation** and **planning**

Traditional
Linear and sequential

Pros and Cons: Waterfall Model

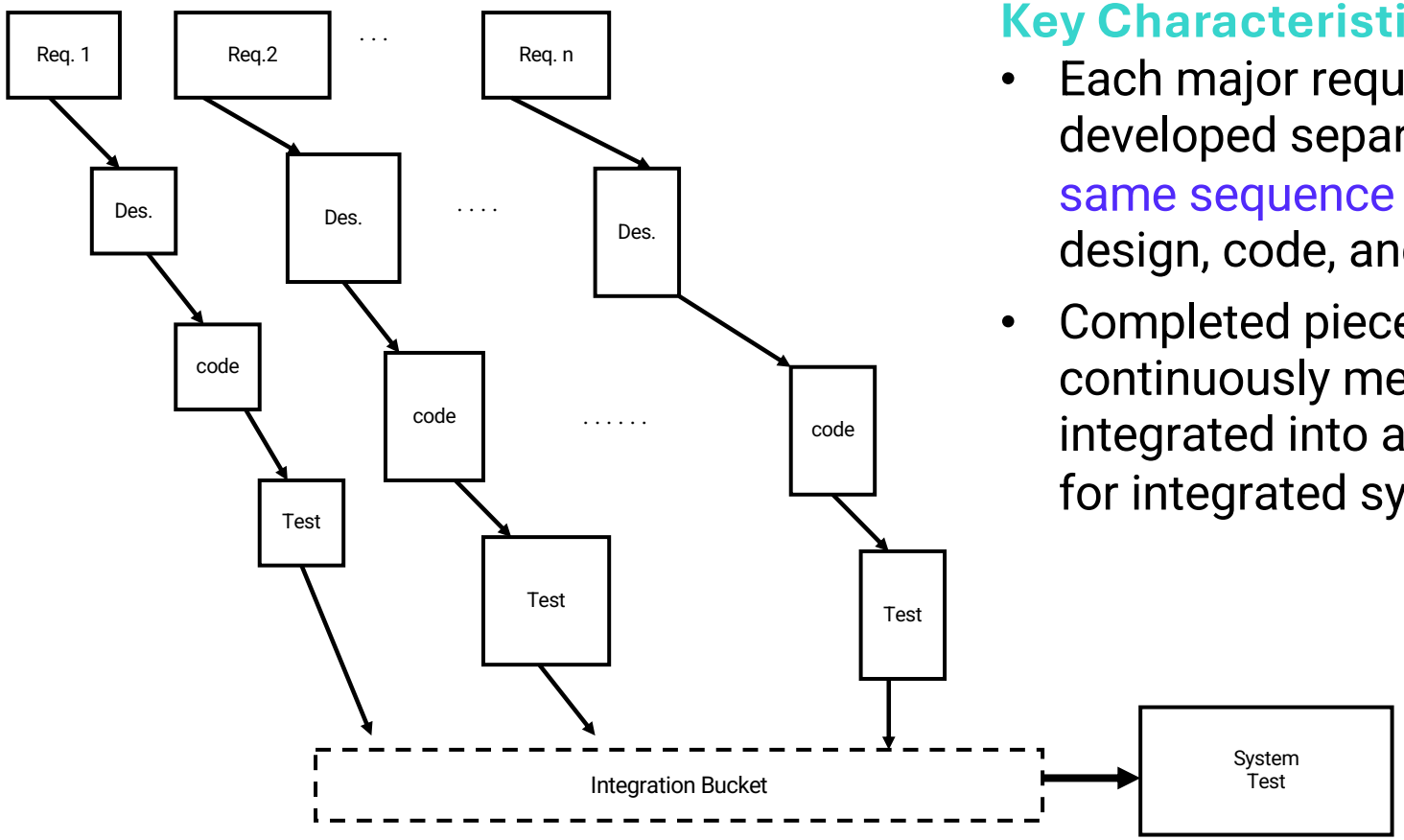
Pros

- Simple and easy to understand
- Well-suited for small projects with clear & fixed requirements
- Strong documentation supports long-term maintenance

Cons

- Lengthy development time
- Inflexible to changes once development begins
- Late discovery of issues (testing comes after coding)
- Poor fit for projects with evolving requirements or high uncertainty

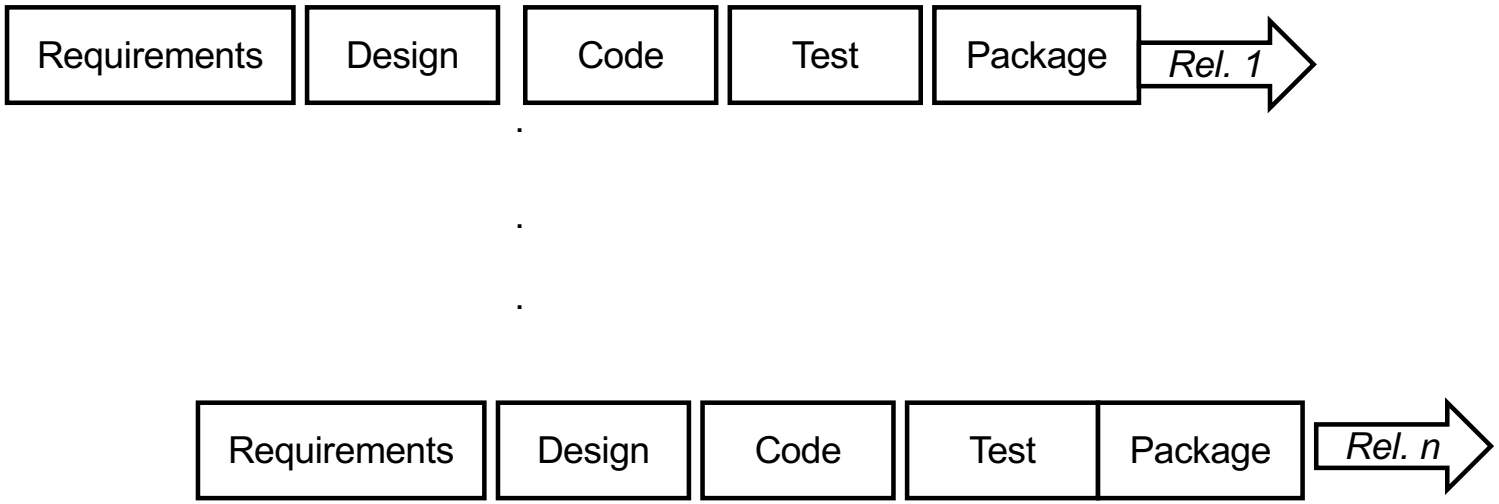
Incremental Model



Key Characteristics:

- Each major requirement/item is developed separately through the same sequence of requirement, design, code, and unit test
- Completed pieces are continuously merged and integrated into a common bucket for integrated system test

Incremental Model B - Multiple Release



Key Characteristic:

- Each small set of requirements is developed, packaged, and released in a multiple release fashion

Pros and Cons: Incremental Model

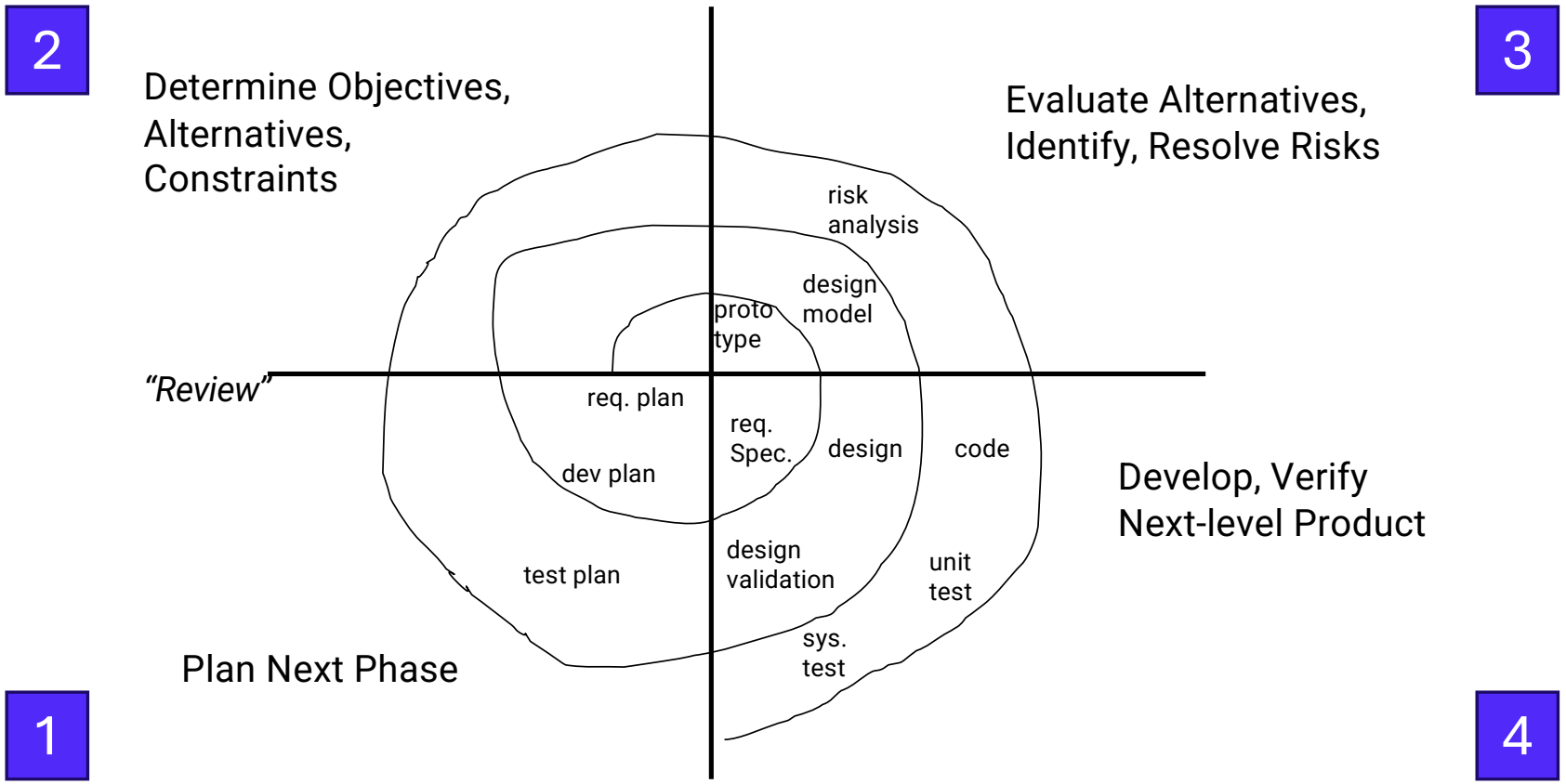
Pros

- Divide and conquer
- Suitable for a very large complex system

Cons

- Parallel execution
- May require more management efforts
- Integration challenges

Spiral Model



Pros and Cons: Spiral Model

Pros

- Require prototyping and modeling
- Iterative and evolutionary approach to all activities based on the number of risks involved
- Can revise things developed in an earlier increment

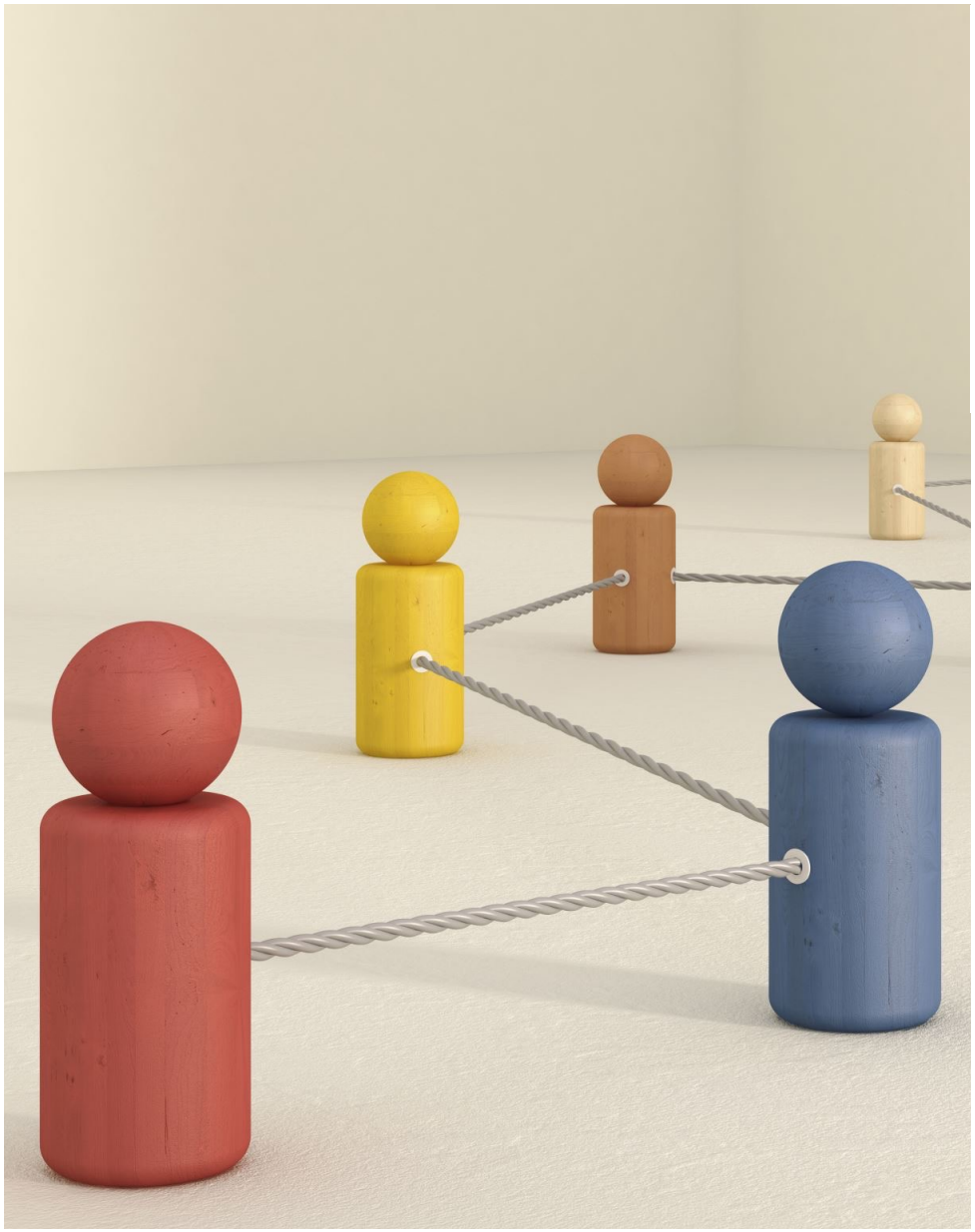
Cons

- Require expertise in risk assessment

Agile

- Agile Manifesto
  - *Individuals and interactions* over processes and tools
  - *Working software* over comprehensive documentation
  - *Customer collaboration* over contract negotiation
  - *Responding to change* over following a plan

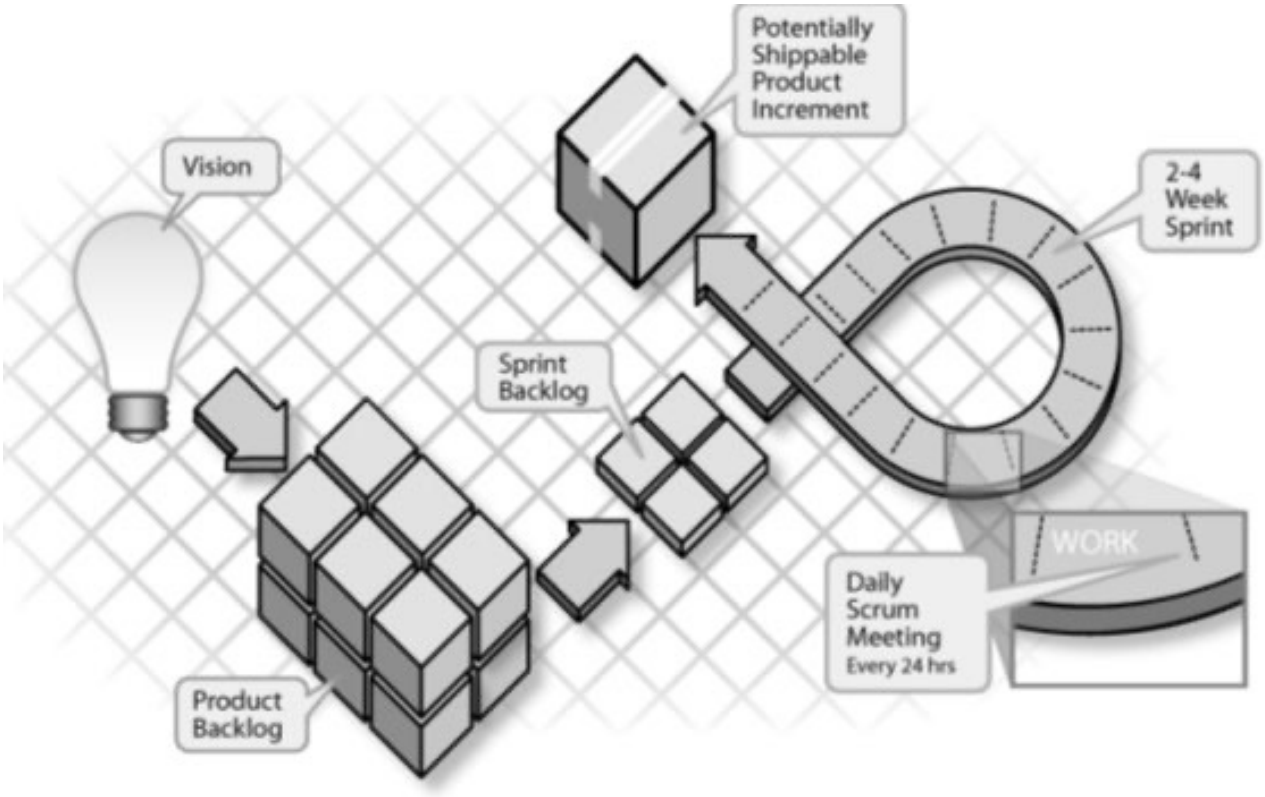
Reference: http://agilemanifesto.org



Agile Processes

- Family of software development methodologies
- Short releases and iterations
- Incremental design
- User involvement
- Minimal documentation
- Informal communications
- Change

Scrum



Key Characteristics:

- Iterative development
- Incremental delivery
- Customer collaboration
- Responding to change
- Cross-functional teams
- Minimal documentation

Reference: http://agilemanifesto.org; https://www.pm-partners.com.au/insights/the-agile-journey-a-scrum-overview/

Pros and Cons: Agile

Pros

- High flexibility and adaptability
- Faster delivery of functional software
- Greater customer satisfaction through regular feedback
- Early detection of problems

Cons

- Requires experienced and disciplined teams
- Less predictable costs and timelines
- Can be challenging for large, fixed-scope projects
- Documentation may be insufficient if not properly managed

Agile vs. Traditional

	Agile	Traditional / Heavy
Requirements	• Assumes change • Informal requirements • Constant user interaction	• Assumes no change • Complete, detailed, formal requirements document
Design	• Informal • Iterative	• Formal • Upfront
User involvement	• Crucial • Frequent	• Beginning (Requirements) • End (Acceptance testing)
Documentation	• Minimal, only as needed • Source code	• heavy, formal documents
Communication	• Informally • Throughout the project	• Documents • Formal memos and meetings
Complexity	• Low	• High
Overhead	• Low	• High



Case Study: Spotify's "Squad" Model

- Wanted to launch new features quickly in a competitive market
- Adopted an Agile approach with a twist:
  - **Squads:** small, autonomous, cross-functional teams owning a specific feature area
  - **Tribes**: groups of related squads for shared direction
  - Heavy emphasis on **continuous delivery**, A/B testing, and rapid feedback



Case Study: Spotify's "Squad" Model

- Why it worked?
  - **Fast iteration** - features went from concept to release in weeks
  - **Autonomy** - squads could adapt their work without waiting for top-down approval
  - **Data-driven decisions** - real-time user analytics informed priorities

Case Study: HealthCare.gov

- Wanted to launched **HealthCare.gov** as a central portal for Americans to sign up for health insurance
- Claimed to use an Agile approach:
  - Multiple contractors working on different modules
  - Frequent meetings and progress tracking

Case Study: HealthCare.gov

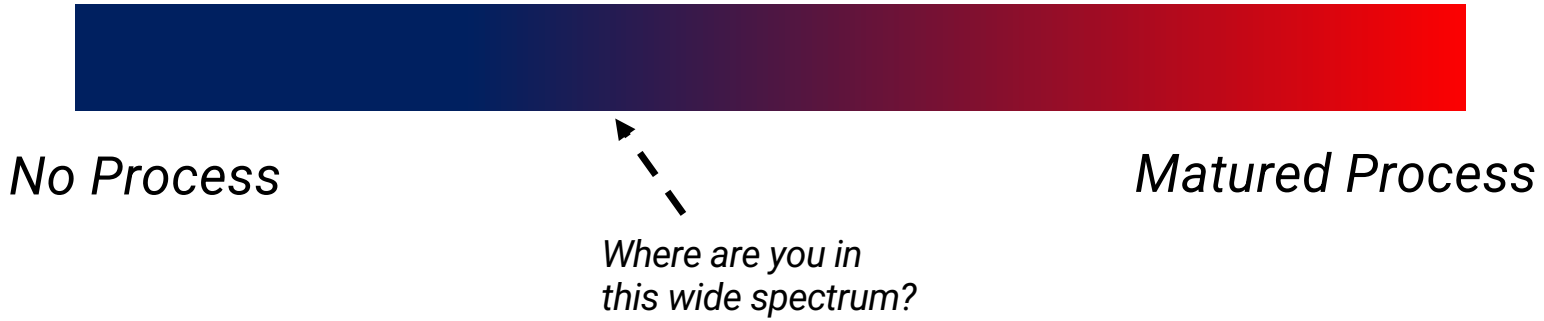
- Why it failed?
  - **Fake Agile** - they kept rigid deadlines and massive scope like Waterfall, but without the coordination discipline
  - **Poor integration** - components didn’t work together when assembled
  - **Late testing** - system testing happened weeks before launch
  - **No real user feedback cycles** - public couldn’t try it until the official launch day

Key comparison

Factor	Spotify success	HealthCare.gov failure
Iteration & Delivery		
User Feedback		
Team Autonomy		
Integration Testing		

Process Assessment

- How mature is your software engineering organization and do you need to improve?
- ISO (ISO 9000 series) and SEI (at Carnegie Mellon) are two leading organizations that help in the **process assessment.**



Online resources

- Software development lifecycle
  - https://www.youtube.com/watch?v=Fi3_BjVzpqk&pp=ygUeU29mdHdhcmUgRGV2ZWxvcG1lbnQgTGlmZWN5Y2xl
  - https://www.youtube.com/watch?v=i-QyW8D3ei0
- Agile methodology
  - https://www.youtube.com/watch?v=8eVXTylZ1Hs
- Scrum
  - https://www.youtube.com/watch?v=XU0llRltyFM